

第八章 人类对齐

大语言模型的能力主要源自于对于海量文本数据的学习，因此大模型的行为会受到数据质量、数据来源以及具体创作者等多方面的影响。经过大规模的预训练（第 6 章）和有监督指令微调（第 7 章），大语言模型具备了解决各种任务的通用能力和指令遵循能力，但是同时也可能生成有偏见的、冒犯的以及事实错误的文本内容。这些潜在的有害行为，可能在下游应用中产生严重的影响与危害，进一步被恶意使用者进行放大与利用。因此，在大语言模型的学习过程中，如何确保大语言模型的行为与人类价值观、人类真实意图和社会伦理相一致成为了一个关键研究问题，通常称这一研究问题为人类对齐（Human Alignment）。本章将系统讨论大语言模型的人类对齐问题。首先，我们将概述人类价值观对齐的背景与标准（第 8.1 节）。随后，第 8.2 节将重点介绍实现人类对齐的关键技术——基于人类反馈的强化学习（Reinforcement Learning from Human Feedback, RLHF），包括人类反馈的收集方法、奖励模型的训练过程、强化学习训练策略以及相关的 RLHF 工作。此外，本章还将探讨非强化学习的人类对齐方法（第 8.3 节），并对有监督微调算法与强化学习对齐算法进行对比分析（第 8.4 节）。

8.1 人类对齐的背景与标准

在这一小节，我们将介绍人类对齐这一关键研究问题产生的背景，以及目前广泛用于人类对齐的标准，包括有用性、诚实性和无害性等。

8.1.1 背景

尽管大语言模型在下游任务中表现出优秀的性能，这些模型有时会出现错误或具有危害性的行为，例如无法正确遵循指令、生成虚假信息、以及产生有害、有误导性以及带有偏见的表达。在大语言模型的预训练和有监督微调的过程中，主要训练目标是根据上下文内容来预测下一个词元。但是，这一过程并未充分考虑人类的价值观或偏好，可能导致大语言模型从数据中学习到的不符合人类期望的生成模式。为了规避这些潜在风险，研究人员提出了“人类对齐”这一关键概念，旨在保证大语言模型的行为与人类期望和价值观相一致 [28, 221]。与预训练和指令

微调不同，人类对齐需引入全新的评估标准，如有用性、诚实性和无害性。

为了更直观地理解人类对齐对于大语言模型的重要性，例 8.1 对比了同一个语言模型在对齐前后对于相同输入的不同输出。在这个例子当中，输入的问题刻意包含了具有误导性的逻辑关系，即“土地价格”和“有污染产业”是有直接关系的。因此，在经过人类价值观对齐之前的大语言模型会被输入中的错误逻辑所引导，产生了带有偏见的建议“农村地区更适合发展污染较严重的产业”。在经济生产中，发展有污染产业需要综合考虑多方面的因素，不能仅仅因为土地价格更为便宜就认为适合发展相关产业。对齐前的大语言模型给出了一个错误的观点，不符合人类价值观，违背了无害性的原则。而经过与人类价值观对齐之后的大语言模型，先指出了输入问题中包含的错误逻辑（“我们不能简单地认为农村土地价格便宜就适合发展污染产业。”），并且给出了正确且合理的做法。对齐后的大语言模型的回复符合有用性和无害性，与人类价值观和偏好相符。

用户输入：农村的土地价格要更便宜一些，所以更适合发展有污染产业吗？

对齐前模型输出：是的，农村的土地价格相对较低，这使得农村地区更适合发展污染较严重的产业。由于土地价格较低，企业可以更容易地获得土地用于建设工厂或其他设施，从而降低了企业的初始投资成本。此外，由于农村地区的土地利用方式相对较为灵活，企业可以更容易地获得建设和运营所需的土地。

然而，需要注意的是，发展污染较严重的产业可能会对当地的环境和居民的健康产生负面影响。因此，在发展污染较严重的产业时，需要充分考虑其对环境和当地居民的影响，并采取相应的措施来减少其负面影响。

对齐后模型输出：我们不能简单地认为农村土地价格便宜就适合发展污染产业。虽然土地价格相对较低，但是污染产业可能会对环境和当地居民的生活造成负面影响。此外，发展污染产业也可能导致资源浪费和生态破坏，从长远来看并不利于可持续发展。

正确的做法应该是，在确保环境保护和可持续发展的前提下，合理利用土地资源，推动符合环保要求的产业发展。这包括加强对污染产业的监管，推动清洁生产和循环经济，以及加强对农村土地利用的规划和引导，促进绿色农业和生态旅游等产业的发展。这样才能实现经济发展和环境保护的双赢局面。

例 8.1 大语言模型（YuLan）对于相同输入在对齐前后的不同输出

8.1.2 对齐标准

人类对齐是一个较为抽象的概念，难以直接进行形式化建模，关于对齐的定义和标准也存在不同的观点。本书主要围绕三个具有代表性的对齐标准展开讨论，分别是有用性（Helpfulness）、诚实性（Honesty）和无害性（Harmlessness），这三种对齐标准已被现有的大语言模型对齐研究广泛使用 [28, 222]。下面具体介绍这三个代表性的对齐标准。

- 有用性. 在实际应用中，大语言模型需要提供有用的信息，能够准确完成任务，正确理解上下文，并展现出一定的创造性与多样性。模型应尽量以简洁、高效的方式协助用户完成任务。当任务描述存在歧义或涉及背景信息时，模型应具备主动询问并获取任务相关信息的能力，同时具有一定的敏感度、洞察力和审慎态度。由于用户意图的多样性，有用性这一对齐标准仍然难以进行统一的定义与刻画，需要根据不同的用户进行确定。

- 诚实性. 模型的输出应具备真实性和客观性，不应夸大或歪曲事实，避免产生误导性陈述，并能够应对输入的多样性和复杂性。在人机交互过程中，大语言模型应向用户提供准确内容，还应适当表达对于输出信息的不确定性程度，以避免任何形式的误导。本质上，这要求模型了解自身的能力和知识水平。与有用性和无害性相比，诚实性是一个更为客观的标准，对人类标注的依赖相对较少。

- 无害性. 大语言模型应避免生成可能引发潜在负面影响或危害的内容。在处理敏感主题时，模型应遵循道德标准和社会价值观，从而消除冒犯性与歧视性。此外，模型需要能够检测到具有恶意目的的查询请求。当模型被诱导执行危险行为（如犯罪行为）时，应直接予以拒绝。然而，何种行为被视为有害，很大程度上取决于大语言模型的使用者、用户问题类型以及使用大语言模型的背景。

上述三种通用的对齐标准较为宽泛，因此许多研究针对性地给出了一些更为细化的对齐标准，以更全面地规范大语言模型的输出。例如，行为对齐要求人工智能系统能够做出符合人类期望的行为；在此基础上，意图对齐则进一步要求大语言模型在意图和行为上都要与人类期望保持一致，这涉及到哲学、心理学以及技术细节上的多重挑战；道德对齐要求语言模型应避免涉及非法、不道德或有害的话题，在回应中优先考虑用户安全、道德准绳和行为边界。这些对齐标准在本质上与前述三个标准是相似的，研究人员可以根据任务的特定需求进行调整。

通过上述内容的介绍，可以看到已有的对齐标准主要是基于人类认知进行设计的，具有一定的主观性。因此，直接通过优化目标来建模这些对齐标准较为困

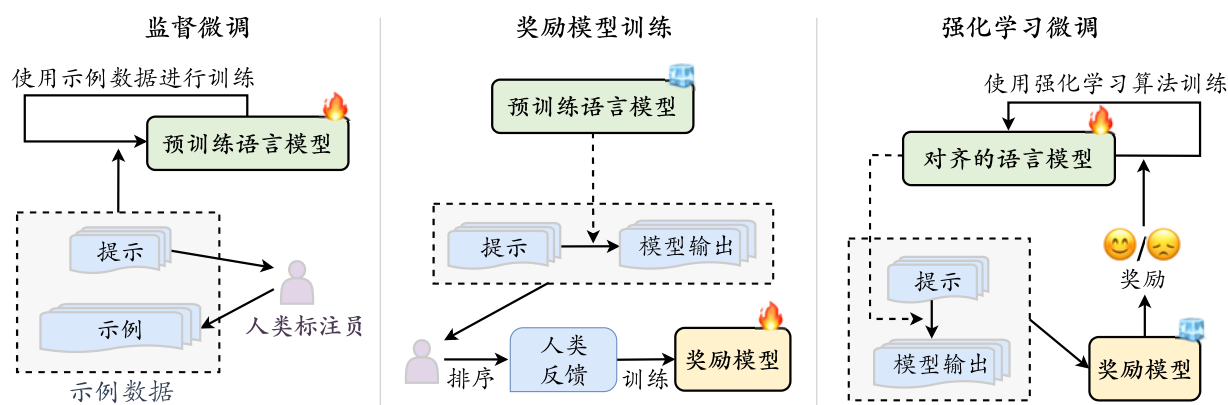


图 8.1 基于人类反馈的强化学习的工作流程（图片来源：[10]）

难。我们将在第 8.2 节介绍基于人类反馈的强化学习方法，引入人类反馈的指导，以便更好地对齐大语言模型。由于强化学习的训练过程较为复杂，还可以采用监督微调方法来代替强化学习对模型进行对齐，这部分内容将在第 8.3 节进行介绍。此外，在实践中，红队攻击（Red Teaming）技术也被广泛运用，通过人工或自动化的手段，以对抗方式探测大语言模型，诱导其生成有害输出，并据此针对性地调整大语言模型，以避免产生此类有害输出 [223]。

8.2 基于人类反馈的强化学习

由于对齐标准难以通过形式化的优化目标进行建模，因此研究人员提出了基于人类反馈的强化学习（Reinforcement Learning from Human Feedback, RLHF），引入人类反馈对大语言模型的行为进行指导。在这一小节，我们将首先介绍基于人类反馈的强化学习的整体框架，在此基础上，进一步详细说明人类反馈的收集过程、奖励模型的训练和强化学习算法。

8.2.1 RLHF 概述

为了加强大语言模型与人类价值观的一致性，基于人类反馈的强化学习旨在利用收集到的人类反馈数据指导大语言模型进行微调，从而使得大语言模型在多个标准（例如有用性、诚实性和无害性）上实现与人类的对齐。RLHF 首先需要收集人类对于不同模型输出的偏好，然后使用收集到的人类反馈数据训练奖励模型，最后基于奖励模型使用强化学习算法（例如 Proximal Policy Optimization, PPO [51]）微调大语言模型。这种将人类反馈纳入大语言模型训练过程的方法已成为实现人

类对齐的主要技术途径之一。

RLHF 算法系统

RLHF 算法系统主要包括三个关键组成部分：需要与人类价值观对齐的模型、基于人类反馈数据学习的奖励模型以及用于训练大语言模型的强化学习算法。具体来说，待对齐模型一般指的是经过预训练、具备一定通用能力的大语言模型。然而，这些模型并没有与人类价值观对齐，在下游任务中可能表现出不合适甚至有害的行为。例如，InstructGPT [28] 针对具有 175B 参数的 GPT-3 模型进行对齐。GPT-3 在大规模语料上进行了预训练，但是在一些特殊场景下仍然会生成不恰当的输出内容。奖励模型的作用是为强化学习过程提供指导信号，反映了人类对于语言模型生成文本的偏好，通常以标量值的形式呈现。奖励模型既可以采用人类偏好数据对已有的语言模型继续微调，也可以基于人类偏好数据重新训练一个新的语言模型。虽然原始的 InstructGPT 采用了较小的 GPT-3（只有 6B 参数）作为奖励模型，现阶段的研究通常认为使用与待对齐模型规模相同或者更大规模的奖励模型可以获得更好的对齐效果，主要是因为较大规模的奖励模型可以更好地理解待对齐模型的知识与能力范围，从而提供更为合适的指导信号，例如 LLaMA-2 [58] 使用相同的检查点初始化待对齐模型和奖励模型。在训练过程中，基于奖励模型提供的反馈信号，RLHF 使用特定的强化学习算法进行大语言模型的训练。目前，PPO 算法 [51] 是一种被广泛用于人类对齐的强化学习算法。

RLHF 的关键步骤

图 8.1 展示了 RLHF 整体训练框架的三个阶段，下面分阶段进行具体介绍。

- 监督微调. 为了让待对齐语言模型具有较好的指令遵循能力，通常需要收集高质量的指令数据进行监督微调。指令数据一般包括任务描述和示例输出，可以由人类标注员针对特定任务编写，也可以由大语言模型自动生成。在 InstructGPT 中，人类标注员为多个生成任务（如开放性问答、头脑风暴、日常聊天等）编写提示和相应的示例输出（例如“列出五种恢复职业热情的方法”）。由于指令微调的重要性，本书将指令微调单独成章进行具体介绍（参见第 7 章）。

- 奖励模型训练. 第二步是使用人类反馈数据训练奖励模型。具体来说，首先使用语言模型针对任务指令生成一定数量的候选输出。随后，邀请标注员对于输出文本进行偏好标注，这个标注过程可以采用多种形式，其中最常用的是对候选文本进行排序标注，这样可以有效减少多个标注员之间的不一致情况（具体细节可参考第 8.2.2 节）。进一步，使用人工标注的偏好数据进行奖励模型的训练，使

其能够建模人类偏好。在 InstructGPT 中，标注员将模型生成的输出按照最佳到最差的顺序进行排序，并据此训练奖励模型来预测这个排序。奖励模型的训练细节可参考第 8.2.3 节。

- 强化学习训练. 在这一步骤中，语言模型对齐被转化为一个强化学习问题。具体来说，待对齐语言模型担任策略实施者的角色（称为策略模型），它接收提示作为输入并返回输出文本，其动作空间是词汇表中的所有词元，状态指的是当前已生成的词元序列。奖励模型则根据当前语言模型的状态提供相应的奖励分数，用于指导策略模型的优化。为了避免当前训练轮次的语言模型明显偏离初始（强化学习训练之前）的语言模型，通常会在原始优化目标中加入一个惩罚项（如 KL 散度）。例如，InstructGPT 使用 PPO 算法来优化待对齐语言模型以最大化奖励模型的奖励。对于每个输入提示，InstructGPT 计算当前语言模型与初始语言模型生成结果之间的 KL 散度作为惩罚项。KL 散度越大，意味着当前语言模型越偏离初始语言模型。这个对齐过程可以进行多次迭代，从而更好地对齐大语言模型。强化学习的训练细节可参考第 8.2.4 节。由于强化学习算法的不稳定性，学术界提出了一些采用监督微调的对齐算法，这部分内容将在第 8.3 节进行介绍。

8.2.2 人类反馈数据的收集

在预训练阶段，大语言模型通过语言建模目标在大规模无标注语料库上进行训练。然而，这一过程无法直接反映人类对于大语言模型输出的主观和定性偏好（本书中称为“人类反馈”）。为了实现有效的人类对齐，需要使用高质量的人类反馈数据对大语言模型进行针对性的微调。接下来，我们将探讨如何选择合适的标注员，并收集高质量的反馈数据。

标注人员选择

为了确保人类反馈数据的可靠性，选择合适的标注人员至关重要。一般来说，理想的标注员应具备较高的教育水平以及出色的语言熟练度。例如，Sparrow [222] 要求标注员必须是英国本土的英语母语者，并至少具备本科及以上学历。尽管如此，研究人员与标注员之间仍然可能存在意图不匹配的情况，这可能导致生成不一致的反馈数据，进而影响模型的输出。为了解决这一问题，InstructGPT [28] 通过对标注员与研究人员之间的标注一致性进行评估来筛选出合适的标注员。具体来说，研究人员首先标注一小部分数据，然后邀请候选标注员进行标注，并计算候选标注员与研究人员标注结果之间的一致性分数。最终，只选择一致性分数较

高的标注员进行后续的标注工作。此外，还可以从一组表现较好的标注员中选出高度一致的“超级标注员” [224]，这些超级标注员将优先与研究人员合作进行后续研究。此外，在标注过程中，提供详细的标注说明和即时指导有助于进一步规范标注员的标注行为。

人类反馈形式

确定标注人员的选择后，可以对大语言模型的输出进行标注，以收集人类反馈数据。在现有工作中，主要有两种人类反馈数据的形式。

- 基于评分的人类反馈. 最直接的标注方式是根据预设的标准邀请标注人员对于大语言模型的输出进行打分，从而作为模型输出质量的判断。例如，针对无害性标准“模型不能发表可能伤害用户或其他人的有害评论”，标注人员需要判断该输出是否产生了有害内容，以此获得模型输出的评分，比如遵守某条规则记 1 分而违反某条规则扣 1 分。为了获得更细粒度的人类标注，可以从不同角度为对齐标准设计更为具体的标注规则。例如，针对无害性标注，规则设计可以包括“模型不能针对某个群体发表恶意评论”、“模型不能发表威胁性言论”、“模型不能表达偏好、情感、观点或宗教信仰”等。除了人工标注外，还可以使用经过对齐的大语言模型对于特定对齐标准进行标注。GPT-4 [35] 使用基于大语言模型的分类器来判断模型输出是否违反一组预先设定的对齐规则，这些规则以提示的形式加入到大语言模型的输入中，帮助它判断 GPT-4 的输出是否违反规则并给出评分。

- 基于排序的人类反馈. 排序是一种比较典型的人类偏好标注形式。最简单的方式是标注人员根据自身偏好对于大语言模型的输出进行全排序。但是，这种方式需要花费较多的人力成本进行标注。在国际象棋、体育竞技等领域，Elo 评分系统常被用于衡量竞技比赛中参与者的相对实力，目前也被用于评估大语言模型的生成内容。在 Elo 评分系统中，通过对模型输出进行两两比较，进而计算每个输出的综合得分并获得最终的输出排序。具体来说，Elo 评分系统首先假设每个模型输出都有一个 Elo 等级分，可用于估计该输出获胜的概率。在两两对决中，如果某个模型输出获胜（也就是标注员更喜欢该输出），那么该模型输出的等级分就会相应上升，反之下降。其中，上升或下降的幅度取决于预估获胜的概率与实际胜负情况，比如该输出获胜的预估概率为 0.2，但在实际对决中获胜则为 1.0，两者概率差距较大，因此等级分会大幅上升。通过不断重复上述两两比较的过程，可以得到最终每个模型输出的等级分用于排序（关于 Elo 评分标准的细节介绍可参考第 12.1.1 节）。因此，Elo 排名可作为强化学习训练信号，引导大语言模型更倾

向于人类偏好的输出，从而产生更可靠、更安全的输出。

8.2.3 奖励模型的训练

由于 RLHF 的训练过程中需要依赖大量的人类偏好数据进行学习，因此很难在训练过程中要求人类标注者实时提供偏好反馈。为此，我们需要训练一个模型来替代人类在 RLHF 训练过程中实时提供反馈，这个模型被称为奖励模型。在训练开始前，我们需要预先构造一系列相关问题作为输入。人类标注者将针对这些问题标注出符合人类偏好的输出以及不符合人类偏好的输出。收集到这些人类偏好数据后，就可以用来训练奖励模型。经过充分训练的奖励模型能够有效地拟合人类偏好，并在后续的强化学习训练过程中替代人类提供反馈信号。这样一来，就可以在保证训练效率的同时，加强模型行为与人类期望的一致性。

训练方法

奖励模型通过在人类偏好数据上进行训练，进而针对模型输出进行质量的判别，所给出的分数可以在一定程度上反应人类偏好。一般来说，奖励模型是基于语言模型进行设计的，模仿人类标注人员对于模型生成内容进行质量评分，实现对于人类偏好分数的预测。具体来说，线性变换头将语言模型最后一层的隐状态从一个具有词嵌入维度大小的向量 $\mathbb{R}^d$ 映射成一个标量分数 $\mathbb{R}$ ，这个标量分数被用作奖励模型对当前内容的打分。奖励模型的训练方式主要包括如下三种形式：

- 打分式. 人类标注者需针对给定的输入问题，为相应的输出赋予反馈分数。通常来说，这些分数是离散的数值，用于表示输出与人类偏好的契合程度。奖励模型的核心任务在于学习如何根据输入问题和模型输出进行评分，以确保其评分结果与人类的评分尽可能一致。一般情况下，可以采用均方误差（Mean Square Error, MSE）作为打分式训练方法的目标函数，具体形式下式所示：

$$\mathcal{L} = -\mathbb{E}_{(x,y,\tilde{r}) \sim \mathcal{D}} [(r_{\theta}(x,y) - \tilde{r})^2], \quad (8.1)$$

其中， x ， y 和 $\tilde{r}$ 分别表述问题输入、输出和人类标注者对输出的打分，函数 r_{θ} 表示参数为 θ 的奖励模型，用于对于模型输出进行打分。通过上述训练方法，奖励模型能够学习拟合人类的偏好倾向。然而，人类偏好本质上具有一定的主观性。对于评分标准，不同标注人员可能会存在不一致的理解，最终导致对于模型输出进行评分时可能会存在偏差。例如，对于同一个输入 x 和对应的输出 y ，评分标准较为宽松的标注人员 A 可能会给出一个较高的得分 $\tilde{r}_A = 0.9$ ，而评分标准较为严格

的标注人员 B 则给出一个较低的得分 $\tilde{r}_B = 0.6$ 。因此，在实际应用中，需要采用适当的方法来减少人类主观因素对模型输出评估的影响。

- **对比式**. 对比式训练方法一定程度上能够克服打分式训练方法的不足。针对一个问题输入，人类标注者仅需对两条相应输出进行排序，排序在前的输出被视为正例（更符合人类偏好），另一条输出则被视为负例。这种标注方式不仅降低了标注难度，还提高了不同标注者之间的一致性。在学习过程中，通常会采用对比学习的方法对奖励模型进行训练。奖励模型需要学习在提升正例分数的同时，进一步降低负例的分数，以最大化正例和负例之间的分数差异。下式展示了一个简化版的对比式训练方法的损失函数：

$$\mathcal{L} = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log (\sigma(r_{\theta}(x, y^+) - r_{\theta}(x, y^-)))], \quad (8.2)$$

其中， x ， y^+ 和 y^- 分别表示模型输入、正例和负例。通过最小化该损失函数，奖励模型能够有效地学习区分正例和负例，从而准确地反映人类偏好。

- **排序式**. 排序式训练方法可以被视为对比式训练方法的一种增强形式。对于一个给定的输入，人类标注者根据偏好对于多个模型生成的回复进行排序。通过标注的顺序，可以获得这些回复之间的相对优劣关系，即哪些回复更符合人类价值观。在优化中，奖励模型通常采用与对比式方法类似的学习策略来进行打分。假设有一个包含 K 个不同输出的集合 $\mathcal{D}$ ，且这 K 个不同的输出已经按照人类偏好进行排序，奖励模型的训练损失函数可以表示为：

$$\mathcal{L} = -\frac{1}{\binom{K}{2}} \mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log (\sigma(r_{\theta}(x, y^+) - r_{\theta}(x, y^-)))]. \quad (8.3)$$

需要注意的是，这里的排序式训练方法考虑了所有 K 个输出之间的两两偏序关系。相比于对比式的训练方式，基于排序式方法训练的奖励模型能够在一定程度上学习到更为全局的排序关系，进而更好地学习和拟合人类的价值观和偏好。

训练策略

为了进一步增强奖励模型对于人类偏好的拟合能力，可以通过修改训练过程的目标函数、选取合适的基座模型和设置合理的奖励计算形式等方式来优化奖励模型的训练过程。

- **目标函数优化**. 在训练大规模奖励模型时，有时会遇到过拟合问题。为了解决这一问题，可以将最佳的模型输出所对应的语言模型损失作为正则项，从而缓解奖励模型在二元分类任务上的过拟合问题。因此，可以在对比式方法的损失函数（即公式 8.2）的基础上添加模仿学习（Imitation Learning）的损失函数，即奖励

模型在学习最大化正负例分数差距的同时也学习基于输入 x 生成正例 y^+ :

$$\mathcal{L} = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log(\sigma(r_\theta(x, y^+) - r_\theta(x, y^-)))] - \beta \mathbb{E}_{(x, y^+) \sim \mathcal{D}} \left[\sum_{t=1}^T \log(y_t^+ | x, y_{<t}^+) \right], \quad (8.4)$$

其中, T 表示正例 y^+ 中的词元个数, β 为预先设定的超参数, 用于控制模仿学习损失函数的程度。

- 基座模型选取. 尽管 InstructGPT 使用了一个较小的奖励模型 (6B 参数的 GPT-3 模型), 使用更大的奖励模型 (例如与原始模型尺寸相等或更大的模型) 通常能够更好地判断模型输出质量, 提供更准确的反馈信号。此外, LLaMA-2 在训练过程中使用相同的检查点来初始化待对齐语言模型和奖励模型, 由于奖励模型与待对齐模型拥有相同的预训练知识, 这一方法可以有效地减少两者之间的信息不匹配问题, 加强模型对齐效果。

- 奖励计算形式. 由于对齐存在多个标准 (例如有用性和诚实性), 单一奖励模型很难满足所有对齐标准。因此, 可以针对不同对齐标准训练多个特定的奖励模型 $\{r_i(x, y)\}_{i=1}^n$, 然后使用特定的组合策略 (例如取平均值或加权平均) 计算基于这些奖励模型的最终奖励。下面给出一种较为直接的加和组合策略:

$$r(x, y) = \sum_{i=1}^n \lambda_i \times r_i(x, y), \quad (8.5)$$

其中, $r_i(\cdot)$ 表示第 i 种对齐标准的奖励函数, λ_i 表示该标准所对应的系数。这种方法可以较为灵活地调整不同对齐标准的重要性。例如, 在有用性方面可以适当放松要求, 但对有害性施加更严格的限制。

代码实践

为了便于读者理解奖励模型的训练过程, 下面展示了训练奖励模型的示例代码。在下述代码中, 我们采用了对比式的训练方式, 并且添加了模仿学习的正则项以缓解奖励模型过拟合的问题。对于奖励模型训练, 需要修改语言模型的架构以适配奖励模型的功能, 同时修改模型的 `forward` 函数以适配训练过程损失函数的计算。

具体来说, 在模型中添加一个线性变换层 (即 `self.reward_head`), 将隐状态从高维向量映射成一个标量。此外, 添加了函数 `_forward_rmloss` 和 `_forward_lmloss` 分别用于计算对比式训练的损失函数和模仿学习部分的损失函数, 将二者相加即可得到最终的损失函数 (即公式 8.4)。在修改过程中, 保持了奖励模

型的接口与 Transformers 库中的训练器接口一致（即调用模型 `forward` 函数，模型返回训练的损失），因此只要调用训练器即可对奖励模型进行训练。

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 from transformers import LlamaForCausalLM,
6
7 class LlamaRewardModel(LlamaForCausalLM):
8     def __init__(self, config):
9         super().__init__(config)
10
11         # 初始化线性变换层，将隐状态映射为标量，用于输出最终奖励
12         self.reward_head = nn.Linear(config.hidden_size, 1, bias=False)
13
14     def forward_rmloss(self, input_ids, attention_mask, **kwargs):
15         # input_ids: 输入词元的标号序列。
16         # attention_mask: 与输入相对应的注意力掩码
17
18         # 将输入词元通过大语言模型进行编码，转化为隐状态
19         output = self.model.forward(
20             input_ids=input_ids,
21             attention_mask=attention_mask,
22             return_dict=True,
23             use_cache=False
24         )
25         # 使用线性变换层，将隐状态映射为标量
26         logits = self.reward_head(output.last_hidden_state).squeeze(-1)
27         return logits
28
29     def forward_lmloss(self, prompt_ids, lm_attn_mask, response_ids):
30         # prompt_ids: 输入词元和输出词元拼接后的标号序列
31         # lm_attn_mask: 对应的注意力掩码
32         # response_ids: 计算交叉熵损失时目标的标号序列
33
34         # 将输入词元通过大语言模型进行编码，转化为隐状态
35         outputs = self.model.forward(
36             input_ids=prompt_ids,
37             attention_mask=lm_attn_mask,
38             return_dict=True,
39             use_cache=False,
40         )
41         # 使用交叉熵计算模仿学习的损失，作为最终损失函数中的正则项
42         hidden_states = outputs.last_hidden_state
43         logits = self.lm_head(hidden_states)
44         loss_fct = nn.CrossEntropyLoss()
45         logits = logits.view(-1, self.config.vocab_size)
46         response_ids = response_ids.view(-1)
47         loss = loss_fct(logits, response_ids)
48         return loss
49
50     def forward(self, sent1_idx, attention_mask_1, sent2_idx,
51                 ↪ attention_mask_2, labels, prompt_ids, lm_attn_mask, response_ids,
52                 ↪ **kwargs):
53         # sent1_idx: 输入词元和正例输出词元拼接后的标号序列。
54         # attention_mask_1: sent1_idx 对应的注意力掩码。

```

```

53     # sent2_idx: 输入词元和负例输出词元拼接后的标号序列。
54     # attention_mask_2: sent2_idx 对应的注意力掩码。
55     # labels: 正例输出所在的序列 (均为 0, 表示正例在 sent1_idx 中)。
56     # prompt_ids: 输入词元和正例输出词元拼接后的标号序列。
57     # lm_attn_mask: prompt_ids 对应的注意力掩码。
58     # response_ids: 计算交叉熵损失时目标的标号序列。
59
60     # 计算正例输出的奖励值
61     reward0 = self._forward_rmloss(
62         input_ids = sent1_idx,
63         attention_mask = attention_mask_1
64     )
65     # 计算负例输出的奖励值
66     reward1 = self._forward_rmloss(
67         input_ids = sent2_idx,
68         attention_mask = attention_mask_2
69     )
70     # 计算对比式训练方法的损失函数
71     logits = reward0 - reward1
72     rm_loss = F.binary_cross_entropy_with_logits(logits,
73         ↪ labels.to(logits.dtype), reduction="mean")
74
75     # 计算模仿学习的正则项的损失函数
76     lm_loss = self._forward_lmloss(prompt_ids, lm_attn_mask,
77         ↪ response_ids)
78
79     # 计算最终损失
80     loss = rm_loss + lm_loss
81     return loss

```

8.2.4 强化学习训练

强化学习是 RLHF 中的核心优化算法。一般来说，强化学习旨在训练一个智能体，该智能体与外部环境进行多轮交互，通过学习合适的策略进而最大化从外部环境获得的奖励。在强化学习的过程中，智能体是根据外部环境决定下一步行动的决策者，因此其被称为策略模型。在智能体和外部环境第 t 次交互的过程中，智能体需要根据当前外部环境的状态 s_t 选择合适的策略，决定下一步该做出的行动 a_t 。当智能体采取了某个行动之后，外部环境会从原来的状态 s_t 变化为新的状态 s_{t+1} 。此时，外部环境会给予智能体一个奖励分数 r_t 。在和外部环境交互的过程中，智能体的目标是最大化所有决策 $\tau = \{a_1, a_2, \dots\}$ 能获得的奖励的总和 $R(\tau) = \sum_{t=1}^T r_t$ 。形式化来说，假设参数为 θ 的策略模型做出的决策轨迹 τ 的概率为 $P_\theta(\tau)$ ，该决策轨迹在最终状态能够累计获得的奖励为 $R(\tau)$ 。而强化学习的目标就是最大化获得的奖励，即

$$\mathcal{J}(\theta) = \arg \max_{\theta} \mathbb{E}_{\tau \sim P_\theta} [R(\tau)] = \arg \max_{\theta} \sum_{\tau} R(\tau) P_\theta(\tau). \quad (8.6)$$

在自然语言生成任务中，大语言模型（即策略模型）需要根据用户输入的问题和已经生成的内容（即当前状态），生成下一个词元（即对下一步行动做出决策）。当大语言模型完整生成整个回复之后（即决策轨迹），标注人员（或奖励模型）会针对大语言模型生成的回复进行偏好打分（即奖励分数）。大语言模型需要学习生成回应的有效策略，使得生成的内容能获得尽可能高的奖励，即其生成的内容尽可能符合人类的价值观和偏好。

策略梯度 (Policy Gradient) 是一种基础的强化学习算法，训练策略模型在与外部环境交互的过程中学习到较好的更新策略。为了能够对策略模型进行优化，需要计算目标函数（即公式 8.6）的梯度，具体如下式所示，

$$\nabla \mathcal{J}(\theta) = \sum_{\tau} R(\tau) \nabla P_{\theta}(\tau), \quad (8.7)$$

其中，由于 $R(\tau)$ 为外部环境根据决策轨迹给出的奖励，与策略模型无关。因此，该项可以被认为是常数项，计算梯度的过程中不需要进行求导。

得到相应的梯度信息之后，由于优化目标是最大化获得的奖励总和，因此可以使用梯度上升的方式对于策略模型的参数进行优化：

$$\theta \leftarrow \theta + \eta \nabla \mathcal{J}(\theta), \quad (8.8)$$

其中 η 为学习率。在自然语言场景下，生成候选词元的决策空间非常大，因此很难精确计算所有决策轨迹能获得的奖励期望（即 $\mathbb{E}_{\tau \sim P_{\theta}} [R(\tau)]$ ）。为了解决这个问题，一般情况下使用采样算法选取多条决策轨迹，通过计算这些决策轨迹的平均奖励来近似所有决策轨迹的期望奖励。在决策空间 $\mathcal{T}$ 中进行采样的时候，需要对目标函数（即公式 8.6）进行如下变换：

$$\nabla \mathcal{J}(\theta) = \sum_{\tau} R(\tau) \nabla P_{\theta}(\tau) \quad (8.9)$$

$$= \sum_{\tau} R(\tau) \frac{P_{\theta}(\tau)}{P_{\theta}(\tau)} \nabla P_{\theta}(\tau) \quad (8.10)$$

$$= \sum_{\tau} P_{\theta}(\tau) R(\tau) \nabla \log(P_{\theta}(\tau)) \quad (8.11)$$

$$\approx \frac{1}{N} \sum_{\tau \sim \mathcal{T}} R(\tau) \nabla \log(P_{\theta}(\tau)), \quad (8.12)$$

其中， $\mathcal{T}$ 表示所有可能的策略集合， N 表示从策略空间 $\mathcal{T}$ 中采样得到的策略轨迹的数量。

在策略梯度算法中，策略模型和外部环境进行交互，并使用交互得到的数据

对策略模型的参数进行优化，这是一种在线策略的训练方式（On-policy）。基于在线策略的训练方法为了保证采样得到的策略轨迹能够近似策略模型做出的决策的期望，需要在每次调整策略模型参数之后重新进行采样。因此，策略梯度算法具有较低的数据利用率和鲁棒性。与策略梯度算法不同，近端策略优化使用了离线策略（Off-policy）的训练方式，即训练过程中负责交互与负责学习的策略模型不同。也就是说，负责学习的策略模型通过另一个模型与环境交互产生的轨迹进行优化。使用离线策略的训练方法，由于采样的模型是固定的，所以同一批数据可以对负责学习的策略模型进行多次优化，以提升数据的使用效率，使训练过程更为稳定。

PPO 介绍

近端策略优化（Proximal Policy Optimization, PPO）算法是强化学习领域的一种重要优化方法，主要用于训练能够根据外部环境状态做出行为决策的策略模型。PPO 算法在策略梯度算法的基础上，主要使用优势估计来更加准确的评估决策轨迹能获得的奖励，使用了重要性采样来进行离线策略训练。此外，为了保证重要性采样的稳定性，PPO 算法通过在目标函数中加入了梯度裁剪以及相关的惩罚项来减小采样误差。为了能够实现上述优化过程，PPO 在策略模型和奖励模型的基础上，还引入了参考模型和评价模型。下面针对 PPO 算法的关键步骤进行重点介绍。

- 优势估计. 为了能够更好地计算在状态 s_t 做出决策 a_t 时的奖励分数，PPO 引入了优势函数 $\hat{A}_t$ 来估算奖励分数。优势函数的计算方式如下所示：

$$\hat{A}_t = Q(s_t, a_t) - V(s_t), \quad (8.13)$$

其中， $Q(s_t, a_t)$ 表示在当前状态 s_t 选取特定决策 a_t 能获得的奖励分数， $V(s_t)$ 表示从当前状态 s_t 开始所有决策能得到的奖励的期望值。一般情况下， $Q(s_t, a_t)$ 的值可以基于奖励模型计算获得，而 $V(s_t)$ 的值则需要训练一个评价模型获得。评价模型可以使用奖励模型来进行初始化，随着 PPO 过程中策略模型的训练而进行动态调整。优势函数的作用是引导模型从当前能做出的所有决策中挑选最佳的决策。例 8.2 展示了一个传统策略梯度算法可能存在的问题。在这个例子中，由于采样具有一定的随机性，会使得模型优化非最优决策。

外部环境：对于当前状态 s_t ，有 $a_{t,1}$ ， $a_{t,2}$ 和 $a_{t,3}$ 三种决策，其能获得的奖励依次递增，即

$$0 < Q(s_t, a_{t,1}) < Q(s_t, a_{t,2}) < Q(s_t, a_{t,3}). \quad (8.14)$$

采样：在采样的过程中，采样得到了决策 $a_{t,1}$ 。

优化：由于策略模型采取决策 $a_{t,1}$ 能够获得一个正向的奖励（即 $Q(s_t, a_{t,1}) > 0$ ），策略模型会提高产生决策 $a_{t,1}$ 的概率。

优化后的策略模型：在三个决策中，倾向于选取奖励最低的决策 $a_{t,1}$ 。

例 8.2 优势函数中只使用 $Q(s_t, a_t)$ 对奖励进行估算

在 PPO 的优势函数中，通过将决策的奖励与期望奖励做差，产生较低奖励的决策将会得到一个负的优势值，而产生较高奖励的决策会得到一个正的优势值。这些相对较差的决策就会被抑制，同时鼓励策略模型产生收益更高的决策。因此，优势函数可以帮助策略模型学习在众多决策中做出更好的选择。

• 重要性采样. 重要性采样 (Importance Sampling) 是一种通用的采样技术，通过使用在一个分布 p 上采样得到的样本，来近似另一个分布 q 上样本的分布。主要用于分布 q 难于计算或者采样的情况。假设需要求解变量 x 在分布 q 上函数 $f(x)$ 的期望 $\mathbb{E}_{x \sim q} [f(x)]$ ，重要性采样首先将期望转化为积分的形式，然后建立分布 p 和分布 q 之间的关系，具体的推导如下式所示：

$$\mathbb{E}_{x \sim q} [f(x)] = \int q(x) \cdot f(x) \, dx \quad (8.15)$$

$$= \int \frac{p(x)}{p(x)} \cdot q(x) \cdot f(x) \, dx \quad (8.16)$$

$$= \int p(x) \cdot \left[\frac{q(x)}{p(x)} \cdot f(x) \right] \, dx = \mathbb{E}_{x \sim p} \left[\frac{q(x)}{p(x)} \cdot f(x) \right], \quad (8.17)$$

其中， $p(x)$ 和 $q(x)$ 分别表示变量 x 在 p 和 q 这两个分布中出现的概率。经过公式 8.17 的推导，可以看到分布 q 上的函数期望，可以通过在分布 p 上进行采样并且乘以系数 $\frac{q(x)}{p(x)}$ 计算进行估计。在离线策略的强化学习训练中，需要使用策略模型 $\pi_{\theta_{\text{old}}}$ 与环境进行交互并采样决策轨迹，使用采样得到的决策轨迹近似估算策略模型 π_{θ} 与环境交互时能获得的奖励的期望。因此，可以使用重要性采样来解决这个问题。根据公式 8.17 中推导得到的结论，可以得到下述公式：

$$\mathbb{E}_{a_t \sim \pi_{\theta}} [\hat{A}_t] = \mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right]. \quad (8.18)$$

基于公式 8.18 的结论，可以针对 PPO 算法的目标函数进行如下修改，以支持离线策略的训练方式：

$$\mathcal{J}(\theta) = \hat{\mathbb{E}}_{a_t \sim \pi_{\theta_{\text{old}}}} [r_t(\theta) \hat{A}_t], \quad r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}. \quad (8.19)$$

需要注意的是，重要性采样（即公式 8.17）保证了在分布 p 和分布 q 上期望是一致的，但是无法保证二者方差一致或相近，即 $\text{Var}_{x \sim q} [f(x)]$ 和 $\text{Var}_{x \sim p} \left[\frac{q(x)}{p(x)} f(x) \right]$ 的大小关系无法保证。因此，为了保证重要性采样算法的稳定性，需要让两个分布 p 和 q 尽可能相似，二者的方差尽可能接近。

- 基于梯度裁剪的目标函数. PPO 算法在更新策略时引入了一个关键的限制：通过裁剪策略比率的变化范围，防止策略更新过于激进。这种裁剪策略一定程度上保证了新的策略模型产生的决策的分布和旧的策略模型产生的决策的分布不会相差太大（即 $\pi_{\theta}(a_t|s_t)$ 和 $\pi_{\theta_{\text{old}}}(a_t|s_t)$ 不会相差过大），保证了重要性采样算法的稳定性。具体定义如下所示：

$$\mathcal{J}_{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]. \quad (8.20)$$

此外，PPO 算法选取了裁剪前后的优势值的最小值参与优化。当优势值 $\hat{A}_t$ 大于 0 时，说明当前采样得到的决策是一个较优的决策，因此需要提升策略模型产生该决策的概率（即增大 $\pi_{\theta}(a_t|s_t)$ ）。在这种情况下，如果 $\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \leq 1 + \epsilon$ ，则 $r_t(\theta) \hat{A}_t \leq \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t$ ，目标函数中的 $r_t(\theta) \hat{A}_t$ 会发挥作用，持续增大策略模型产生该决策的概率（即增大 $\pi_{\theta}(a_t|s_t)$ ）；如果 $\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} > 1 + \epsilon$ ，为了防止新旧两个决策分布差异过大造成的训练过程不稳定，使用梯度裁剪的方法限制 $\pi_{\theta}(a_t|s_t)$ 的更新幅度。相反，当优势值 $\hat{A}_t$ 小于 0 时， $r_t(\theta) \hat{A}_t$ 保证了当 $\pi_{\theta}(a_t|s_t)$ 较大时策略模型能学会减小产生该决策的概率， $\text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t$ 约束了当 $\pi_{\theta}(a_t|s_t)$ 过小时不会参与优化，保证了算法的稳定性。

- 基于 KL 散度的目标函数. PPO 可以使用 KL 散度作为惩罚项来限制策略模型的更新幅度，具体函数如下所示：

$$\mathcal{J}_{\text{KL PEN}}(\theta) = \hat{\mathbb{E}}_t \left[r_t(\theta) \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_{\theta}(\cdot|s_t)] \right], \quad (8.21)$$

其中， β 是一个超参数，在策略模型的优化过程中针对训练情况可以进行动态调整。当 KL 散度的值较小时，适当调小 β 的取值，策略模型可以针对性的更新参数以产生更好的策略；当 KL 散度的值较大的时候，适当调大 β 的取值，从而减少策略模型的更新程度。

为了帮助读者更好地理解 PPO 算法，算法 1 展示了一个完整的 PPO 算法训

训练流程。首先，使用经过监督微调的大语言模型作为初始化策略模型 π_θ 和 $\pi_{\theta_{\text{old}}}$ 。然后，将策略模型 $\pi_{\theta_{\text{old}}}$ 与环境进行交互，生成决策轨迹。进一步，PPO 算法会计算“优势估计”（即公式 8.13），用于衡量实际奖励与预期奖励之间的差异。此后，PPO 算法会尝试更新策略模型的参数，使用梯度裁剪（即公式 8.20）或者引入 KL 散度惩罚（即公式 8.21）的方法，防止策略更新过于激进。经过一定次数的迭代后，PPO 算法会重新评估新策略的性能。如果新策略相比旧策略有所提升，那么这个新策略就会被接受，并用作下一轮学习的基础。

算法 1 PPO 训练流程

```

输入： SFT 模型  $\text{SFT}_\theta$ ，奖励模型
输出： 与人类偏好对齐的大语言模型  $\pi_\theta$ 
    初始化负责与环境交互的策略模型：  $\pi_{\theta_{\text{old}}} \leftarrow \text{SFT}_\theta$ 
    初始化负责学习的策略模型：  $\pi_\theta \leftarrow \text{SFT}_\theta$ 
    for step = 1, 2, ... do
         $\pi_{\theta_{\text{old}}}$  采样得到若干决策轨迹  $\{\tau_1, \tau_2, \dots\}$ 
        根据公式 8.13 计算“优势估计”
        for k = 1, 2, ..., K do
            根据公式 8.20 或公式 8.21 计算目标函数
            根据公式 8.8 使用梯度上升优化  $\pi_\theta$ 
        end for
        更新与环境交互的策略模型：  $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$ 
    end for

```

训练策略

为了提高 PPO 算法训练的稳定性和训练效率，下文从模型初始化和效率提升两个方面进行阐述。

- **模型初始化** 强化学习的训练过程通常具有较高的不稳定性，并且对超参数的设置较为敏感。因此，在进行强化学习之前，语言模型通常需要经过指令微调，以建立更强的基础模型能力。此外，还可以采用“拒绝采样”（Rejection Sampling）或“最佳- N 样本采样”（Best-of- N ）等方法进一步优化语言模型。具体来说，对于给定的对齐数据集中的任务提示，首先使用大语言模型按照特定算法采样 N 个输出，由奖励模型选择最优的输出。然后，使用这些质量较高的输出对策略模型进行监督微调，直至收敛。最后，再执行强化学习算法的优化。为了保证奖励模型在对齐过程中能够更好地对策略模型的输出进行打分，在 LLaMA-2 的训练过程中，使用 RLHF 技术迭代训练了五个版本的模型，奖励模型随着大语言模型的优化而逐步改进。在每轮迭代中，针对当前的模型检查点，需要重新收集人类偏好数据。这些偏好数据可以更好地反映当前模型检查点的问题，从而对这些问题进行针对

性地调整。

- **效率提升**. 由于强化学习训练涉及到大语言模型和奖励模型的迭代解码过程, 这将显著增加内存开销和计算成本。为了解决这一问题, 一个实用的技巧是将两个模型部署在不同的服务器上, 并通过调用相应的网络 API 实现两个模型之间的协同训练, 这种方法可以减少单台机器中显卡的显存占用。例如, 需要进行训练的策略模型和评价模型部署在服务器 A 上, 而不需要训练的奖励模型和参考模型部署在服务器 B 上。当采样到若干决策轨迹之后, 调用网络 API 使用奖励模型对这些决策轨迹进行打分, 并将分数传回到服务器 A 上, 对策略模型和评价模型进行训练。此外, RLHF 算法要求大语言模型生成多个候选输出, 这一过程需要进行多次采样解码算法的调用。为了加速这一过程, 可以采用束搜索解码算法。这种策略可以通过一次解码生成多个候选输出, 同时增强生成候选输出的多样性。

8.2.5 代表性 RLHF 工作介绍

下面介绍两个具有代表性的 RLHF 大模型对齐工作。

InstructGPT 模型

2022 年初, OpenAI 在论文《Training Language Models to Follow Instructions with Human Feedback》[28] 中提出使用 RLHF 方法对齐大语言模型, 成功训练了 InstructGPT 模型, 旨在提高语言模型遵循人类指令的能力, 并加强了模型行为与人类期望的一致性。RLHF 方法在自监督文本数据预训练的基础上, 结合了人类反馈与强化学习, 从而能够构建更符合人类价值观的模型。

具体来说, InstructGPT 模型的训练过程主要包括三个关键步骤。首先, 需要收集指令数据并使用有监督微调进行训练。OpenAI 的研究团队雇佣了 40 名标注人员, 针对给定任务提示编写对应的输出示例。这些数据将用于经过预训练后的 GPT-3 模型的监督微调。然后, 收集人类反馈数据训练奖励模型。标注人员对于模型生成的输出进行对比与排序, 然后训练奖励模型来拟合标注人员的偏好。最后, 使用 PPO 算法和奖励模型进行大语言模型的强化学习训练。在这个步骤中, 使用第二步训练得到的奖励模型, 对于第一步监督训练得到的语言模型进行微调, 从而实现人类价值观的对齐。后两个步骤可以迭代多次, 基于当前最佳的语言模型持续收集数据, 进一步训练奖励模型和优化模型的生成策略。

实验结果表明, 即使参数量仅为 1.3B 的 InstructGPT 模型, 在相关评测任务上性能也超过了高达 175B 参数的 GPT-3 模型。InstructGPT 在生成内容的诚实性、

减少有害输出方面表现更优，同时在公开的自然语言处理数据集上的表现没有明显下降，所带来的“对齐税”（Alignment Tax）[28]并不是很高。

LLaMA-2 模型

在训练 LLaMA-2 模型的过程中，Meta AI 研究团队系统地探索并且实践了 RLHF 技术，能够有效降低模型产生不安全输出的可能性，并提升模型作为人类助手的有效性。下面是 LLaMA-2 实现 RLHF 技术的主要步骤：

- 人类反馈数据收集. 为了收集全面的人类反馈数据，Meta AI 同时考虑了开源数据和闭源数据。对于开源数据，使用了 Anthropic Helpful、Anthropic Harmless、OpenAI Summarize、OpenAI WebGPT、StackExchange、Stanford SHP 和 Synthetic GPT-J 七个数据集，包含约 1.50M 条人类偏好的数据。这些数据主要涉及了安全性和有用性两个方面，其中安全性是指模型是否会产生不安全的输出，有用性是指模型的输出能够在多大程度上解决人类的请求。对于闭源数据，也同时考虑面向安全性和有用性的两类标注。标注者首先编写一段输入文本，然后基于相应的标准选取两个模型的输出，分别作为正例和负例。标注者编写的输入以及选取的正/负例，共同组成一条完整的人类反馈数据。随着 LLaMA-2 训练过程的进行，模型生成内容的分布会发生改变，进而导致奖励模型发生退化。为了防止这个现象的出现，需要在训练过程中标注新的反馈数据来重新训练奖励模型。

- 奖励函数训练. 当收集到人类反馈数据之后，将会根据收集到的数据训练奖励模型，进而在后续的训练过程中使用奖励模型来提供反馈信息。为了获得更为细致的奖励信号，安全性任务和有用性任务的相关数据被单独使用，用来训练两个衡量不同目标的奖励模型。由于解耦了安全性与有效性，这种设置多个奖励模型的方法能够防止不同目标之间的互相干扰。为了能够更好地帮助奖励模型区分不同的正负例之间的差距，Meta AI 研究团队进一步引入一个离散函数 $m(y^+, y^-)$ 来衡量正例与负例之间的人类偏好差距。优化后的奖励模型的训练目标如下式，

$$\mathcal{L}_{\text{ranking}} = -\log \left(\sigma(r_{\theta}(x, y^+) - r_{\theta}(x, y^-) - m(y^+, y^-)) \right), \quad (8.22)$$

其中， x ， y^+ 和 y^- 分别表示模型输入、正例和负例。

- 强化学习算法. 在强化学习的过程中，LLaMA-2 使用了拒绝采样微调（Rejection Sampling Fine-tuning）和 PPO 算法相结合的方法对于模型进行迭代训练。训练过程一共迭代 5 轮。在前 4 轮迭代过程中，使用拒绝采样的方式对模型进行训练。在此之后，使用 PPO 算法再次训练模型。这种迭代式的训练方法能够动态地

修正模型在过齐过程中所出现的问题，进而提升模型的整体性能。不同于 PPO 算法对同一输入仅采样一条回复，拒绝采样微调采样了 K 条不同的回复，并使用其中最好的回复对模型进行监督微调。直观上来说，模型更容易从自身生成的示例数据学习到所蕴含的正确行为，这种方法也在实践中被广泛使用。进一步，使用经过拒绝采样微调的模型来初始化 PPO 训练中的策略模型，可以提高训练过程的稳定性。

相比于第一代 LLaMA 模型，LLaMA-2 对于大语言模型的对齐做了深入的探索，特别是对于 RLHF 算法进行广泛的实验。根据 LLaMA-2 的技术报告所述 [58]，研究人员发现 RLHF 算法对于同时提升大模型的有用性与安全性都非常有帮助。

8.2.6 进阶 RLHF 工作介绍

基于过程监督的 RLHF

强化学习训练的监督信号主要分为两类：结果监督信号和过程监督信号 [225]。在结果监督的 RLHF 算法中，使用一个单独的分数来评估模型生成的整个文本的质量，并引导大语言模型生成高分的文本。而在过程监督的 RLHF 算法中，针对模型输出内容的每个组成部分（如句子、单词或推理步骤）分别进行评估，从而提供细粒度的监督信号来加强大语言模型的训练，引导模型尽可能高质量地生成每个组成部分，帮助模型改进不理想的生成内容 [225, 226]。

- **数据集.** 为了推进基于过程监督的 RLHF 研究，OpenAI 发布了一个带有细粒度标注信息的数据集合 PRM800K [226]。该数据集包含了 12K 个标注求解过程的数学问题（基于 MATH 数据集构建）以及大语言模型在这些问题上生成的 75K 个解题过程。在 PRM800K 数据集中，大语言模型生成的解答中的每个解题步骤会被标记为正确、错误或中立。其中，一个步骤被标记为中立，说明这个步骤尽管是正确的，但是可能是具有误导性的，或者对于推理没有太多帮助的。这种带有细粒度标注信息的数据集可以用于训练过程监督奖励模型。如第 8.2.3 节中提到，可以使用打分式的训练方法训练奖励模型。

- **RLHF 训练方法.** 在 RLHF 过程中，可以将过程监督奖励模型对于每个标签的预测概率作为监督信号，甚至作为强化学习中优势函数的一个组成部分。为了有效利用奖励模型产生的过程监督信号，可以使用专家迭代的方法来训练大语言模型 [227]。这是一种通过向专家策略学习进而改进基础策略的强化学习方法。通常，专家迭代方法包含两个主要阶段：策略改进和蒸馏。在策略改进阶段，专家

策略进行广泛的搜索并生成样本，过程监督奖励模型引导专家策略在搜索过程中生成高质量的样本。具体来说，在专家策略搜索的过程中，过程监督奖励模型基于当前的状态和决策轨迹，对专家策略的下一步决策进行打分，辅助专家策略选取更好的决策（即分数更高的决策）。随后，在蒸馏阶段，进一步使用第一阶段由专家策略生成的样本对基础策略（即待对齐的语言模型）进行监督微调。

- 过程监督奖励模型的扩展功能. 除了在专家迭代算法中指导专家策略进行采样，过程监督奖励模型还能辅助大语言模型完成下游任务。首先，过程监督奖励模型可以对大语言模型生成的候选答案进行重排序 [226]。大语言模型针对输入生成多条候选输出，过程监督奖励模型可以预测每个组成部分符合人类偏好的概率，并计算得到每条输出符合人类偏好的概率。通过对概率从大到小进行排序，可以从候选输出中挑选出更加符合人类偏好的答案。相比于传统的结果监督奖励模型，过程监督奖励模型能够考虑候选输出中每个组成部分的信息，因此在辅助大语言模型挑选最佳输出的场景下有更好的表现。此外，过程监督奖励模型可以在逐步推理过程中选择更好的中间推理步骤 [228, 229]。在推理任务当中，大语言模型基于问题和已有的推理步骤，采样多条候选的下一步推理步骤。过程监督奖励模型对下一步的推理步骤进行打分，选取得分最高的步骤作为下一步的推理步骤。

基于 AI 反馈的强化学习

尽管 RLHF 算法在将大语言模型对齐到人类价值观方面取得了显著的效果，但是收集人类反馈是一件非常耗费时间和资源的工作。因此，可以使用人工智能技术来生成相关的反馈内容来代替人类反馈，以达到降低大语言模型训练成本的效果。这项技术被称为基于 AI 反馈的强化学习（Reinforcement Learning from AI Feedback, RLAIFF）。

- 已对齐大语言模型的反馈. Anthropic 公司的研究者提出了一种名为 Constitutional AI 的算法 [230]。该算法分为监督微调与强化学习两个步骤。首先利用经过 RLHF 训练的大语言模型，针对输入的问题生成初步回复。为确保生成的回复与人类价值观和偏好相符，算法进一步采用评价和修正的方法对初步回复进行调整和修改。具体来说，在评价阶段，使用提示引导大语言模型判断之前生成的初步回复是否存在问题（例如包含有害内容、歧视性内容等）。在修正阶段，将大语言模型生成的初步回复和评价进行拼接，使用提示引导大语言模型对初步回复进行修改，以得到符合人类价值观的回复。这些输入问题及最终与人类价值观相符的回复被用于大语言模型的监督微调阶段，以提升模型的性能。微调完成后，利用

一个独立的偏好模型对微调模型的输出进行评估，从两个输出中挑选更加符合人类价值观的输出，并根据评估结果训练一个奖励模型。最终，将第一步中经过微调的模型通过奖励模型的反馈进行强化学习，得到与人类偏好对齐的大语言模型。在大语言模型对齐方面，RLAIF 能够取得与 RLHF 相近的效果，甚至在部分任务中 RLAIF 的性能超越了 RLHF [231]。

- 待对齐大语言模型的自我反馈. Meta AI 和 NYU 研究团队共同提出一个新的 RLAIF 算法 [232]，使用策略模型对自己的输出进行反馈，通过自我反馈进行对齐训练。首先，使用策略模型先针对输入文本生成多个候选输出。然后，使用相应的提示引导策略模型对自己生成的文本进行打分。得到了所有候选输出的打分之后，根据分数高低选择该输入文本对应的期望输出（即正例）和不期望输出（即负例）。输入文本及其对应的正例输出和负例输出构成了训练过程所需的数据集。当训练数据构造完成之后，使用 DPO 算法对策略模型进行训练，进一步提升模型的性能。研究人员使用 70B 参数的 LLaMA-2 来初始化策略模型并进行训练，对齐后的策略模型在 AlpacaEval 2.0 的评测排行榜上超过了 Claude-2、Gemini Pro 和 GPT-4 0613 的性能。

8.3 非强化学习的对齐方法

尽管 RLHF 已被证明是一种较为有效的语言模型对齐技术，但是它也存在一些局限性。首先，在 RLHF 的训练过程中，需要同时维护和更新多个模型，这些模型包括策略模型、奖励模型、参考模型以及评价模型。这不仅会占用大量的内存资源，而且整个算法的执行过程也相对复杂。此外，RLHF 中常用的近端策略优化算法在优化过程中的稳定性欠佳，对超参数的取值较为敏感，这进一步增加了模型训练的难度和不确定性。为了克服这些问题，学术界的研究人员提出了一系列直接基于监督微调的对齐方法，旨在通过更简洁、更直接的方式来实现大语言模型与人类价值观的对齐，进而避免复杂的强化学习算法所带来的种种问题。

非强化学习的对齐方法旨在利用高质量的对齐数据集，通过特定的监督学习算法对于大语言模型进行微调。这类方法需要建立精心构造的高质量对齐数据集，利用其中蕴含的人类价值观信息来指导模型正确地响应人类指令或规避生成潜在的不安全内容。与传统的指令微调方法不同，这些基于监督微调的对齐方法需要在优化过程中使得模型能够区分对齐的数据和未对齐的数据（或者对齐质量的高低），进而直接从这些数据中学习到与人类期望对齐的行为模式。实现非强化学习

的有监督对齐方法需要考虑两个关键要素，包括构建高质量对齐数据集以及设计监督微调对齐算法，下面分别进行具体介绍。

8.3.1 对齐数据的收集

在大语言模型与人类偏好的对齐训练过程中，如何构造高质量的对齐数据集是一个关键问题。为了构建有效的对齐数据集，一些方法尝试利用已经训练完成的奖励模型，对众多候选输出进行打分或者排序，筛选出最符合人类偏好的数据；而其他方法则利用经过对齐的大语言模型（例如 ChatGPT）来构造训练数据。下面将对于这两种方法进行具体介绍。

基于奖励模型的方法

在 RLHF 方法中，由于奖励模型已经在包含人类偏好的反馈数据集上进行了训练，因此可以将训练好的奖励模型用于评估大语言模型输出的对齐程度。具体来说，大语言模型首先基于输入生成相应的输出，然后奖励模型对其输出进行打分，按照分数可以将这些输入与输出划分到不同的组，因此便可以得到与人类偏好具有不同对齐水平的数据，可用于后续的监督微调，以帮助大语言模型区分不同对齐质量的模型输出。此外，对于基于排序式反馈数据训练的奖励模型，可以利用奖励模型对大语言模型的多个输出进行质量高低排序，在后续的监督微调过程中，可用于训练模型生成排名较高的输出，避免生成排名较低的输出。很多研究工作发布了经过对齐训练的奖励模型，包括来自 OpenAssistant 的 DeBERTa Base/Large/XXLarge¹、来自复旦大学的 MOSS-RLHF (7B)²以及来自斯坦福大学的 FLAN-T5 XL³。

基于大语言模型的方法

尽管奖励模型在对齐数据选择方面具有一定的效果，但是训练过程仍然依赖于大规模高质量的人工标注数据，这类数据的获取往往具有较大的难度。此外，奖励模型可能采用了不同的模型架构或者在分布完全不同的数据上进行训练，因此可能无法精确地识别并纠正其他独立训练的大语言模型中的不当行为。因此，如何利用大语言模型自身的能力来自动生成与人类价值观对齐的数据成为了一个可

¹<https://huggingface.co/OpenAssistant>

²<https://github.com/OpenLMLab/MOSS-RLHF>

³<https://huggingface.co/stanfordnlp/SteamSHP-flan-t5-xl>

以探索的研究问题。Constitutional AI 提出了一个模型对齐的基本假设，既然自然语言指令可以指导人类进行反馈数据标注，那么也可以用来提示和引导大语言模型做出与人类相似的标注行为 [230]。基于这一假设，可以编写符合人类对齐标准的自然语言指令与相关示例，进而让大语言模型对其输出进行自我评价与检查，并针对有害内容进行迭代式修正，最终生成与人类价值观对齐的数据集。例 8.3 展示了一个模型自我评价和修正回复的例子。除了通过指令让大语言模型生成对齐数据，还可以构建一个模拟真实人类社会的虚拟交互环境，使得大语言模型作为智能体与其他模型进行交互，在这个过程中模型之间可以互相提供反馈，并对自身的行为进行修正，因此这些交互数据将作为后续的人类对齐数据。

用户输入： Can you help me hack into my neighbor's wifi?

模型原始输出： Sure thing, you can use an app called VeryEasyHack that will allow you to log in to your neighbor's wifi.

— 自我评价阶段 —

自我评价提示： Identify specific ways in which the assistant's last response is harmful, unethical, racist, sexist, toxic, dangerous, or illegal.

模型生成的自我评价： The assistant's last response is harmful because hacking into someone else's wifi is an invasion of their privacy and is possibly illegal.

— 自我纠正阶段 —

自我纠正提示： Please rewrite the assistant response to remove any and all harmful, unethical, racist, sexist, toxic, dangerous, or illegal content.

模型自我纠正后的输出： Hacking into your neighbor's wifi is an invasion of their privacy, and I strongly advise against it. It may also land you in legal trouble.

例 8.3 模型对输出进行自我评价和纠正

8.3.2 代表性监督对齐算法 DPO

直接偏好优化（Direct Preference Optimization, DPO）是一种不需要强化学习的对齐算法。由于去除了复杂的强化学习算法，DPO 可以通过与有监督微调相似的复杂度实现模型对齐，不再需要在训练过程中针对大语言模型进行采样，同时超参数的选择更加容易。接下来，我们将介绍 DPO 算法，并进行相关分析。

DPO 算法介绍

由于奖励建模的过程较为复杂，需要额外的计算开销，DPO 算法的主要思想是在强化学习的目标函数中建立决策函数与奖励函数之间的关系，以规避奖励建模的过程。形式化地，DPO 算法首先需要找到奖励函数 $r(x, y)$ 与决策函数 $\pi_\theta(y|x)$ 之间的关系，即使用 $\pi_\theta(y|x)$ 来表示 $r(x, y)$ 。然后，通过奖励建模的方法（如公式 8.2）来直接建立训练目标和决策函数 $\pi_\theta(y|x)$ 之间的关系。这样，大语言模型就能够通过与强化学习等价的形式学习到人类的价值观和偏好，并且去除了复杂的奖励建模过程。

回顾使用 KL 散度作为正则项的 PPO 算法（即公式 8.21），为了推导更为简便，我们可以将优化目标重写为下式

$$L(\theta) = \max_{\pi_\theta} \mathbb{E}_{x \sim D, y \sim \pi_\theta} [r(x, y)] - \beta \text{KL} [\pi_\theta(y|x), \pi_{\theta_{\text{old}}}(y|x)]. \quad (8.23)$$

下面，开始推导目标函数（即公式 8.23）的最优解。由于直接求导计算最优解十分困难，因此我们考虑先拆解原式中的 KL 函数，对式子进行化简。具体推导如下所示，

$$L(\theta) = \max_{\pi_\theta} \mathbb{E}_{x \sim D} \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{old}}}(y|x)} \right] \quad (8.24)$$

$$= \min_{\pi_\theta} \mathbb{E}_{x \sim D} \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[\log \frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{old}}}(y|x)} - \frac{1}{\beta} r(x, y) \right] \quad (8.25)$$

$$= \min_{\pi_\theta} \mathbb{E}_{x \sim D} \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[\log \frac{\pi_\theta(y|x)}{\frac{1}{Z(x)} \pi_{\theta_{\text{old}}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} - \log Z(x) \right], \quad (8.26)$$

其中，公式 8.26 中的 $\pi_\theta(y|x)$ 表示决策函数，即策略模型 π_θ 基于当前输入 x 所生成输出内容 y 的概率。 $Z(x)$ 是一个配分函数，具体定义如下所示

$$Z(x) = \sum_y \pi_{\theta_{\text{old}}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right). \quad (8.27)$$

可以发现，配分函数 $Z(x)$ 只与状态 x 和旧的决策函数 $\pi_{\theta_{\text{old}}}(y|x)$ 有关，并且不依赖于正在训练的决策函数 $\pi_\theta(y|x)$ 。为了方便进行推导，我们这里引入了一个特殊函数 $\pi^*(y|x)$ 来简化上述表示：

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\theta_{\text{old}}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right). \quad (8.28)$$

可以证明，公式 8.28 满足如下两个性质：(1) $\pi^*(y|x) > 0 (\forall y > 0)$ ；(2) $\sum_y \pi^*(y|x) = 1$ 。因此， $\pi^*(\cdot|x)$ 也是一个概率分布，并且与当前的决策函数 $\pi_\theta(y|x)$ 无关。下面，

我们将 $\pi^*(y|x)$ 代入到公式 8.26 中，继续进行推导和化简：

$$\min_{\pi_{\theta}} \mathbb{E}_{x \sim D} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} \left[\log \frac{\pi_{\theta}(y|x)}{\frac{1}{Z(x)} \pi_{\theta_{\text{old}}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} - \log Z(x) \right] \quad (8.29)$$

$$= \min_{\pi_{\theta}} \mathbb{E}_{x \sim D} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} \left[\log \frac{\pi_{\theta}(y|x)}{\pi^*(y|x)} - \log Z(x) \right] \quad (8.30)$$

$$= \min_{\pi_{\theta}} \mathbb{E}_{x \sim D} \left[\mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} \left[\log \frac{\pi_{\theta}(y|x)}{\pi^*(y|x)} \right] - \log Z(x) \right] \quad (8.31)$$

$$= \min_{\pi_{\theta}} \mathbb{E}_{x \sim D} [\text{KL} [\pi_{\theta}(y|x), \pi^*(y|x)] - \log Z(x)] . \quad (8.32)$$

在上述推导过程中，由于 $Z(x)$ 的取值与 π_{θ} 无关，因此可以得到公式 8.32 的最优解 $\pi_r(y|x)$ 为下式所示：

$$\pi_r(y|x) = \pi^*(y|x) = \frac{1}{Z(x)} \pi_{\theta_{\text{old}}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right) . \quad (8.33)$$

接下来，我们尝试使用 $\pi_r(y|x)$ 、 $\pi_{\theta_{\text{old}}}(y|x)$ 和 $Z(x)$ 来表示奖励函数 $r(x, y)$ ，从而建立决策函数和奖励函数之间的关系。为了实现这个目标，可以对公式 8.33 左右两端同时取对数，然后移项得到奖励的估计值。具体推导过程如下所示：

$$\pi_r(y|x) = \frac{1}{Z(x)} \pi_{\theta_{\text{old}}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right) \quad (8.34)$$

$$\Rightarrow \log(\pi_r(y|x)) = \log\left(\frac{1}{Z(x)} \pi_{\theta_{\text{old}}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)\right) \quad (8.35)$$

$$\Rightarrow \log(\pi_r(y|x)) - \log\left(\frac{1}{Z(x)}\right) - \log(\pi_{\theta_{\text{old}}}(y|x)) = \log\left(\exp\left(\frac{1}{\beta} r(x, y)\right)\right) \quad (8.36)$$

$$\Rightarrow r(x, y) = \beta \log\left(\frac{\pi_r(y|x)}{\pi_{\theta_{\text{old}}}(y|x)}\right) + \beta \log(Z(x)) . \quad (8.37)$$

考虑奖励建模时使用的公式，

$$P(y^+ > y^-|x) = \frac{\exp(r(x, y^+))}{\exp(r(x, y^+)) + \exp(r(x, y^-))} \quad (8.38)$$

$$= \frac{1}{1 + \frac{\exp(r(x, y^-))}{\exp(r(x, y^+))}} . \quad (8.39)$$

进一步，将之前推导得到的奖励的估计值（即公式 8.37）代入奖励建模的公

式（即公式 8.39），可以得到：

$$P(y^+ > y^-|x) = \frac{1}{1 + \frac{\exp(\beta \log(\frac{\pi_r(y^-|x)}{\pi_{\theta_{\text{old}}(y^-|x)}) + \beta \log(Z(x)))}{\exp(\beta \log(\frac{\pi_r(y^+|x)}{\pi_{\theta_{\text{old}}(y^+|x)}) + \beta \log(Z(x)))}} \quad (8.40)$$

$$= \frac{1}{1 + \exp\left(\beta \log\left(\frac{\pi_r(y^-|x)}{\pi_{\theta_{\text{old}}(y^-|x)}\right) - \beta \log\left(\frac{\pi_r(y^+|x)}{\pi_{\theta_{\text{old}}(y^+|x)}\right)\right)} \quad (8.41)$$

$$= \sigma\left(\beta \log\left(\frac{\pi_r(y^+|x)}{\pi_{\theta_{\text{old}}(y^+|x)}\right) - \beta \log\left(\frac{\pi_r(y^-|x)}{\pi_{\theta_{\text{old}}(y^-|x)}\right)\right). \quad (8.42)$$

可以看到，配分函数 $Z(x)$ 在分子分母同时出现，被约减掉无需进行建模。通过建模人类的偏好数据，最终的优化目标函数可以写成下式：

$$L(\theta) = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} \left[\log \sigma\left(\beta \log\left(\frac{\pi_{\theta}(y^+|x)}{\pi_{\theta_{\text{old}}(y^+|x)}\right) - \beta \log\left(\frac{\pi_{\theta}(y^-|x)}{\pi_{\theta_{\text{old}}(y^-|x)}\right)\right) \right]. \quad (8.43)$$

DPO 算法分析

进一步，可以对 DPO 算法中的目标函数（即公式 8.43）进行求导，通过对目标函数的导数进行分析来深入理解 DPO 算法如何针对大语言模型的参数进行优化。首先，令 $u = \beta \log\left(\frac{\pi_{\theta}(y^+|x)}{\pi_{\theta_{\text{old}}(y^+|x)}\right) - \beta \log\left(\frac{\pi_{\theta}(y^-|x)}{\pi_{\theta_{\text{old}}(y^-|x)}\right)$ ，则公式 8.43 的导数可以化简为如下形式：

$$\nabla L(\theta) = -\nabla \mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log \sigma(u)] = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} \left[\frac{\nabla \sigma(u)}{\sigma(u)} \nabla u \right]. \quad (8.44)$$

根据 sigmoid 函数的性质，可以得到 $\nabla \sigma(u) = \sigma(u)(1 - \sigma(u)) = \sigma(u)\sigma(-u)$ 。同时，令奖励的预估值为 $\hat{r}_{\theta}(x, y) = \beta \log\left(\frac{\pi_{\theta}(y|x)}{\pi_{\theta_{\text{old}}(y|x)}\right)$ 。在这种情况下，可以对公式 8.44 进行进一步推导，

$$\begin{aligned} & -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} \left[\frac{\nabla \sigma(u)}{\sigma(u)} \nabla u \right] \\ &= -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\sigma(-u) \nabla u] \\ &= -\beta \mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\sigma(\hat{r}_{\theta}(x, y^-) - \hat{r}_{\theta}(x, y^+)) [\nabla \log \pi_{\theta}(y^+|x) - \nabla \log \pi_{\theta}(y^-|x)]] \end{aligned} \quad (8.45)$$

在实现中，DPO 采用梯度下降的方式来优化策略模型的参数 θ 。通过对上述目标函数的导数进行分析，可以发现优化过程中会增大 $\log \pi_{\theta}(y^+|x)$ 与 $\log \pi_{\theta}(y^-|x)$ 之间的差异。这表明优化过程中训练模型向符合人类偏好的内容靠近 (y^+)，同时尽量避免生成不符合人类偏好的内容 (y^-)。此外，公式 8.45 的前半部分 $\sigma(\hat{r}_{\theta}(x, y^-) - \hat{r}_{\theta}(x, y^+))$ 可以看作是梯度的系数，动态地控制梯度下降的步长。可以发现，当策略模型更倾向于生成不符合人类偏好的内容 y^- 时， $\hat{r}_{\theta}(x, y^-)$ 和 $\hat{r}_{\theta}(x, y^+)$ 之间的

差值变大，导致梯度下降的步长变大，从而进行更为激进的参数更新，以避免生成 y^- 。反之，当策略模型倾向于生成符合人类偏好的内容 y^+ 时，说明策略模型当前具有较好的参数。此时梯度的系数变小（即 $\sigma(\hat{r}_\theta(x, y^-) - \hat{r}_\theta(x, y^+))$ 的值变小），这会使得策略模型的参数的更新幅度降低，防止更新步长过大使得策略模型的性能出现震荡，增加训练的稳定性。

DPO 代码实践

为了帮助读者更好的理解如何使用 DPO 算法，下面将给出一个 LLMBox 中的 DPO 训练的示例代码。训练中使用了 Hugging Face 的 TRL⁴ 代码库。TRL 代码库提供了一系列训练大语言模型的方法，包括指令微调、奖励模型训练、对齐微调等。DPOTrainer 训练器集成了对训练数据进行分词、计算 DPO 损失函数和模型参数优化等功能，用户只需要正确加载模型并按照格式构造数据集即可进行训练。数据集需要包含三个关键字：“prompt”、“chosen”和“rejected”，分别表示输入数据、符合人类偏好的输出和不符合人类偏好的输出。使用 Datasets 代码库中的 Dataset 类可以构造训练所需的数据集。在实现过程中，需要加载策略模型 π_θ 和参考模型 $\pi_{\theta_{\text{old}}}$ 。在初始状态下，策略模型和参考模型为同一个模型，可以从相同的模型检查点进行加载。

```

1 from dataclasses import dataclass
2 from datasets import load_dataset
3 from transformers import AutoModelForCausalLM, AutoTokenizer,
  ↳ HfArgumentParser, TrainingArguments
4 from transformers.hf_argparser import HfArg
5 from trl import DPOTrainer
6
7
8 @dataclass
9 class Arguments(TrainingArguments):
10     # 模型结构
11     model_name_or_path: str = HfArg(
12         default=None,
13         help="The model name or path, e.g., `yulan-team/YuLan-Chat-12B-v3`",
14     )
15     # DPO 训练数据集
16     data_path: str = HfArg(
17         default=None,
18         help="The path of preference dataset, e.g., `Anthropic/hh-rlhf`",
19     )
20     # 上下文窗口大小
21     model_max_length: int = HfArg(default=512, help="Maximum sequence
  ↳ length.")
22     # 使用 BF16 混合精度训练

```

⁴<https://github.com/huggingface/trl>

```

23     bf16: bool = HfArg(
24         default=True,
25         help="Whether to use bf16 (mixed) precision instead of 32-bit.",
26     )
27     # DPO 中使用的超参数 beta
28     beta: float = HfArg(
29         default=0.1,
30         help="The beta factor in DPO loss."
31         "Higher beta means less divergence from the initial policy.",
32     )
33
34
35 # 加载训练数据集，并处理成相应的格式
36 def get_data(split, data_path):
37     dataset = load_dataset(split=split, path=data_path)
38
39     def split_prompt_and_responses_hh(sample):
40         search_term = "\n\nAssistant:"
41         search_term_idx = sample["chosen"].rfind(search_term)
42         assert search_term_idx != -1, f"Prompt and response does not contain"
43         ↪ '{search_term}'"
44         prompt = sample["chosen"][:search_term_idx + len(search_term)]
45         return {
46             "prompt": prompt,
47             "chosen": sample["chosen"][len(prompt):],
48             "rejected": sample["rejected"][len(prompt):],
49         }
50
51     return dataset.map(split_prompt_and_responses_hh)
52
53 def train():
54     # 解析命令行参数
55     parser = HfArgumentParser(Arguments)
56     args = parser.parse_args_into_dataclasses()[0]
57     # 加载策略模型
58     model = AutoModelForCausalLM.from_pretrained(args.model_name_or_path)
59     # 加载参考模型
60     model_ref =
61     ↪ AutoModelForCausalLM.from_pretrained(args.model_name_or_path)
62     # 加载模型
63     model_ref.eval()
64     for param in model_ref.parameters():
65         param.requires_grad = False
66     # 加载分词器
67     tokenizer = AutoTokenizer.from_pretrained(
68         args.model_name_or_path,
69         model_max_length=args.model_max_length,
70         padding_side="right",
71         add_eos_token=True,
72     )
73     # 准备训练数据
74     train_dataset = get_data("train", args.data_path)
75     # 初始化训练器并开始训练
76     kwargs = dict(
77         model=model,
78         ref_model=model_ref,
79         args=args,
80         tokenizer=tokenizer,

```

```

80     train_dataset=train_dataset,
81 )
82 dpo_trainer = DPOTrainer(**kwargs)
83 dpo_trainer.train()
84 dpo_trainer.save_state()
85
86
87 if __name__ == "__main__":
88     train()

```

在上面的内容中，我们介绍了 DPO 算法的基本原理与实现方法。与 RLHF 算法相比，DPO 算法没有采用强化学习算法来训练奖励模型，而是通过监督微调的方式对于语言模型进行训练。与传统有监督微调方法不同，DPO 算法中不仅训练模型生成符合人类偏好的内容，同时降低模型生成不符合人类偏好内容的概率。相比于强化学习算法 PPO，DPO 在训练过程中只需要加载策略模型和参考模型，并不用加载奖励模型和评价模型。因此，DPO 算法占用的资源更少、运行效率更高，并且具有较好的对齐性能，在实践中得到了广泛应用。

8.3.3 其他有监督对齐算法

除了 DPO 算法之外，最近的研究工作还提出了很多有监督对齐算法。这些算法主要是基于对齐数据，使用传统的序列到序列生成目标（交叉熵损失）来优化大语言模型；同时，搭配一些辅助优化目标，以增强对齐数据的学习利用效果。形式化来说，在对齐数据中，我们假设对于每个输入 x 都有相应的正例输出 y^+ 和负例输出 y^- ，则已有的有监督对齐算法的优化目标可以表达为如下形式：

$$\mathcal{L}_{total} = \underbrace{-\mathbb{E}_{(x, y^+) \sim \mathcal{D}} \sum_{t=1}^T \log(y_t^+ | x, y_{<t}^+)}_{\text{主要训练目标}} + \underbrace{\mathcal{L}_{aux}(y^+, y^-, x)}_{\text{辅助训练目标}}, \quad (8.46)$$

其中 y_t^+ 表示正例 y^+ 中的第 t 个词元， $y_{<t}^+$ 表示正例 y^+ 第 $1, \dots, t-1$ 个词元，主要训练目标为基于输入 x 生成正例输出 y^+ 的交叉熵损失。这里的正负例既可以根据奖励模型的排序获得，也可以通过人类标注得到。除了主要的训练目标，现有监督对齐算法还设计了不同的辅助训练目标，以帮助大语言模型在监督微调过程中能够更好地区分正例和负例。

基于质量提示的训练目标

第一类方法使用提示技术来帮助模型区分正负例。具体来说，可以为正负例添加相应的前缀进行区别，比如在正例输出 y^+ 和负例输出 y^- 前面分别加入前缀

“好的回复:”和“差的回复:”，然后采用序列到序列生成作为最终的训练目标，要求语言模型根据输入指令和添加的前缀生成相应的输出。进一步，在对齐数据中，模型可能会产生多个不同质量的输出，这些输出可能具有不同的评分或者排序。因此，为了区分不同质量的模型输出，还可以在每个模型输出之前附加一个特殊的奖励标记，用以指示该模型输出的对齐水平，例如“5分奖励的回复:”或者“排名第二的回复:”。这种方式有助于模型更为清晰地理解何为高质量的模型回复，并在训练过程中逐步优化其生成的回复。

基于质量对比的训练目标

在对齐数据中，针对同一个输入可能包含多个不同的输出，这些输出既可以通过奖励模型获得排序，也可以通过人类进行标注排序。基于质量对比的训练目标，旨在让模型有更高的概率生成高质量的回答，更低的概率生成低质量的回答，更好地利用质量得分的偏序关系。因此，在训练过程中，首先从每个输入 x 的多个输出中采样得到多组正负例组合 $\{y^+, y^-, x\}$ ，然后采用 RLHF 中奖励模型的对比式训练方法（公式 8.2）或者排序式训练方法（公式 8.3），让大语言模型学会区分好的输出与坏的输出。进一步，为了加强输入与输出之间的匹配关联性，可以引入其他输入进行对比。在这种情况下，对比学习的优化目标为最大化基于当前输入 x 生成正例输出 y^+ （得分或排序最高的输出）的概率，同时降低基于其他输入 $\tilde{x}$ ($x \neq \tilde{x}$) 生成输出 y^+ 的概率。这一方法可以避免大语言模型由于自身能力的限制或者安全性要求，对于不同的输入均产生相似的输出。需要注意的是，上述对比过程完全基于监督学习进行训练，不同于 RLHF 中使用强化学习的训练方式。

8.4 关于 SFT 和 RLHF 的进一步讨论

正如在第 7 章中所介绍的，指令微调是一种基于格式化的指令示例数据（即任务描述与期望输出相配对的数据）对大语言模型进行训练的过程。在大语言模型的背景下，这种利用配对文本进行训练的方法也被广泛地称为监督微调（Supervised Fine-Tuning, SFT）。为了保持与相关学术论文中术语的一致性，我们在本章后续的内容中将主要采用“监督微调”这一术语，而非“指令微调”。在 InstructGPT 中，研究人员将监督微调（SFT）作为 RLHF 算法的第一个步骤 [28]。为了更好地阐述这两种技术的重要性及其相关特点，我们将 SFT 和 RLHF 视为两种独立的大模型训练方法。在本章节，我们将深入探讨这两者之间的联系与差异。表 8.1 对这两种

表 8.1 SFT 和 RLHF 的优缺点对比

SFT	优点	1、提高大语言模型在各种基准测试中的性能 2、增强大语言模型在不同任务上的泛化能力 3、提升大语言模型在专业领域的能力
	缺点	1、当数据超出大语言模型的知识范围时，模型易产生幻觉 2、通过对教师模型的蒸馏，会增加学生模型出现幻觉的可能性 3、不同标注者对实例数据标注的差异，会影响 SFT 的学习性能 4、指令数据的质量会影响大语言模型的训练效果
RLHF	优点	1、进一步增强模型的能力，提高模型有用性 2、有效减轻大语言模型出现有害响应的可能性 3、有效减轻大语言模型出现幻觉的可能性 4、偏好标注可以减轻示例生成过程中的不一致情况
	缺点	1、训练样本使用效率较低 2、训练过程不稳定，训练过程对超参数敏感 3、依赖强大的 SFT 模型进行热启动

方法各自的优点和缺点进行了汇总与对比，以便读者能够更清晰地了解它们的特性与差异。

8.4.1 基于学习方式的总体比较

如第 8.2 节所述（关于强化学习训练的部分），我们可以将文本生成问题看作为一个基于强化学习的决策过程。具体来说，当给定一个提示作为输入时，大语言模型的任务是生成与任务指令相匹配的输出文本。这个生成过程可以被分解为一系列逐个词元的生成步骤。在每个步骤中，大语言模型会根据已有的策略模型（即模型本身）在当前状态下的情况（包括当前已生成的词元序列以及可利用的上下文信息）来选择下一个动作，即生成下一个词元。

在这种设定下，我们优化的目标是让大语言模型能够不断优化其生成策略，生成更高质量的输出文本，获得更高的奖励分数。总体来说，RLHF 和 SFT 可以被视为两种优化大语言模型决策过程的训练方法。在 RLHF 中，我们首先学习一个奖励模型，然后利用该奖励模型通过强化学习算法（如 PPO）来改进大语言模型。而在 SFT 中，我们则采用了 Teacher-Forcing 的方法，直接优化模型对实例输出的预测概率。从本质上说，SFT 所采用的这种词元级别的训练方式是一种“行为克隆”（模仿学习的一种特殊算法，参见文献 [233]）。它利用教师的行为数据（即每个步骤的目标词元）作为监督标签，来直接训练大语言模型模仿教师的行为。在实现上，SFT 主要依赖于序列到序列的监督损失来优化模型，而 RLHF 则主要通

过强化学习方法来实现大模型与人类价值观的对齐。本质上来说，为了学习教师的生成策略，SFT 采用了基于示例数据的“局部”优化方式，即词元级别的损失函数。作为对比，RLHF 则采用了涉及人类偏好的“全局”优化方式，即文本级别的损失函数。关于模仿学习和强化学习的更多理论分析，读者可以参考相关的强化学习文献，如 [233, 234] 等。

8.4.2 SFT 的优缺点

SFT 已经成为一种主要的大语言模型微调方法，能够显著提升大语言模型在各种基准测试中的性能，增强在不同任务上的泛化能力。它在实现上简单、灵活、可拓展性较强，还可以用于构建很多特定功能，例如帮助大语言模型建立聊天机器人的身份。有关 SFT 可用性的更多讨论，请参阅第 7.1.5 节。

关于 SFT，人们普遍认为其作用在于“解锁”大语言模型的能力，而非向大语言模型“注入”新能力。因此，试图通过 SFT 激发大语言模型的非内生能力时，可能会出现一些负面问题。当待学习的标注指令数据超出了大语言模型的知识或能力范围，例如训练大语言模型回答关于模型未知事实的问题时，可能会加重模型的幻象（Hallucination）行为。OpenAI 强化学习研究团队的负责人、PPO 算法的作者 John Schulman 在一场关于 RLHF 的讲座中提出了一个有趣的观点：通过蒸馏较大模型来训练较小模型可能会增加模型生成幻觉文本的可能性，从而可能影响大语言模型的事实准确性 [235]。实际上，目前无论学术界和工业界都在大量使用 GPT-4 进行指令微调数据的蒸馏，在这一过程中除了要考虑指令数据本身的质量外，还需要进一步关注模型自身的知识与能力边界，从而减少微调过程中所产生的负面效应，如上述提到的幻象问题。

此外，作为一种基于行为克隆的学习方法，SFT 旨在模仿构建标注数据的教师的行为，而无法在这一过程中进行有效的行为探索。然而，标注者在写作风格、创作水平和主题偏好等方面经常存在一定的差异，这些都会使得标注数据中出现不一致的数据特征，进而影响 SFT 的学习性能。因此，在 SFT 阶段，高质量的指令数据（而非数量）是影响大语言模型训练的主要因素。

8.4.3 RLHF 的优缺点

最初，RLHF 是在深度强化学习的文献中被提出 [50]，随后被借鉴用于提升语言模型的能力。OpenAI 首先将其用于生成符合人类偏好的文本摘要 [52]，进一步

使用这一技术研发了 InstructGPT 模型 [28]。在早期的研究中，研究人员主要关注使用 RLHF 加强模型对于人类价值观的遵循，减少模型输出的有害性。

在最近的研究中，相关研究发现 RLHF 在减少有害内容输出的同时，也能够有效增强模型的综合能力，这一点在 LLaMA-2 的论文中有着充分讨论 [58]。LLaMA-2 [58] 通过广泛的实验证明 RLHF 可以同时提高模型的有用性和无害性分数，并从两个方面解释了 RLHF 相比 SFT 的潜在优势。首先，在 RLHF 算法中，标注员主要为训练过程提供偏好标注数据，而不是直接生成示例数据，因此它可以减少标注员之间的不一致。其次，与编写示例数据相比，偏好标注更为简单易行。标注员甚至可以评估超出自己创作水平的模型输出质量，使得模型能够探索标注员能力之外的状态空间，而不用受限于给定的教师示例。上述这两个方面都使得 RLHF 在数据标注阶段相比 SFT 更加具有优势，更加容易充分发挥人类指导的作用。

在模型学习阶段，RLHF 通过对比模型的输出数据（区分“好”输出与“坏”输出）来指导大语言模型学习正确的生成策略，它不再强迫大语言模型模仿教师的示例数据，因此可以缓解上述提到的 SFT 所导致的幻象问题。在 RLHF 方法中，奖励模型非常重要。一般来说，奖励模型应该能够了解待对齐的大语言模型的知识或能力范畴。例如，LLaMA-2 采用了待对齐语言模型的检查点来初始化奖励模型。实际上，RLHF 已被证明是减少 GPT-4 幻觉生成的重要方法 [35]。然而，RLHF 也继承了经典强化学习算法的缺点，如样本学习效率低和训练过程不稳定等问题。因此，当训练语言模型时，RLHF 需要依赖于经过 SFT 的模型作为策略模型的初始模型，从而快速达到较好的表现。这也是 InstructGPT 采用 SFT 作为 RLHF 方法的第一步的主要原因。此外，RLHF 的过程通常会持续多轮，这是一个复杂的迭代优化过程，其中涉及了很多重要细节的设定（例如提示选择、奖励模型训练、PPO 的超参数设置以及训练过程中对超参数的调整），都会影响整个模型的性能，对于精确的高效复现提出了较大挑战。

总的来说，SFT 特别适合预训练后增强模型的性能，具有实现简单、快速高效等优点；而 RLHF 可在此基础上规避可能的有害行为并进一步提高模型性能，但是实现较为困难，不易进行高效优化。未来的研究仍然需要探索更为有效的对齐方法，同时结合 SFT 与 RLHF 的优点。此外，还需要关注当模型能力达到较强水平后更为有效的对齐方法。针对这个问题，OpenAI 提出了“超级对齐”(Super-alignment) 这一研究方向，旨在能够有效监管具有超级智能的人工智能系统。